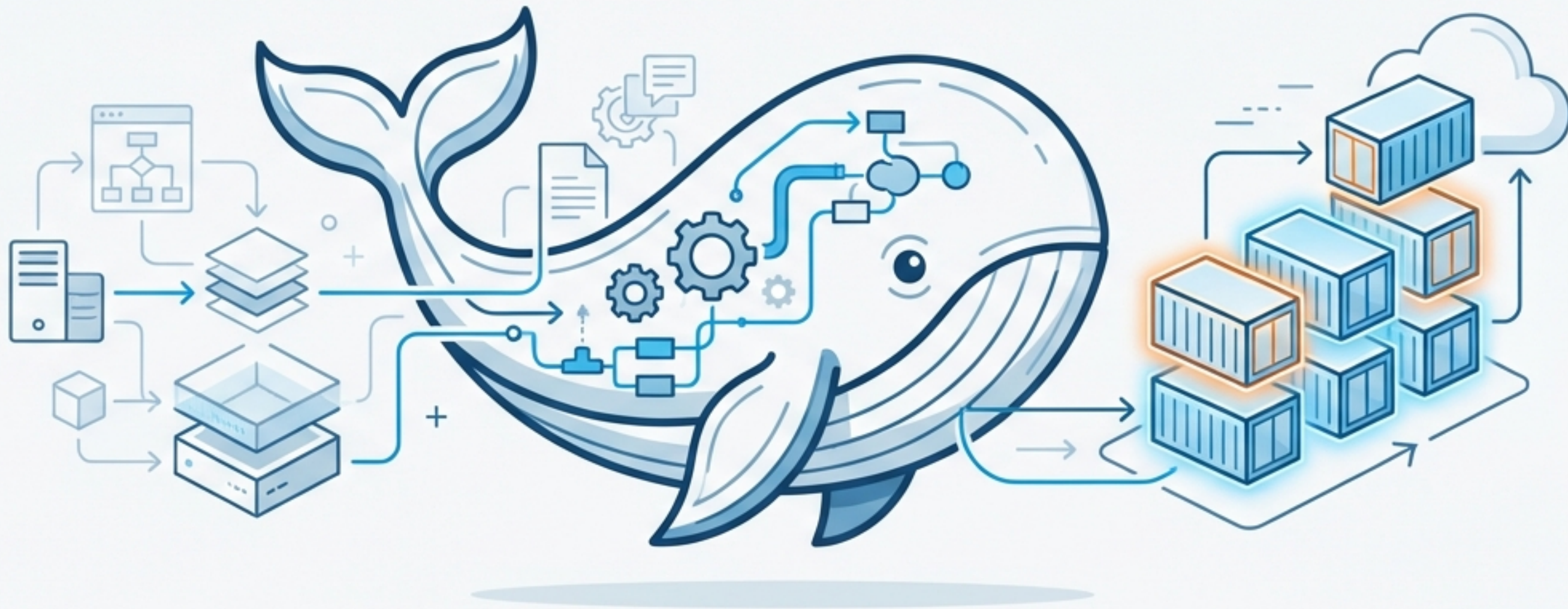


Docker : De la Théorie à la Pratique

Maîtriser les conteneurs, de la virtualisation aux déploiements agiles.



Théorie & Virtualisation
Roboto Slab (for any minination)

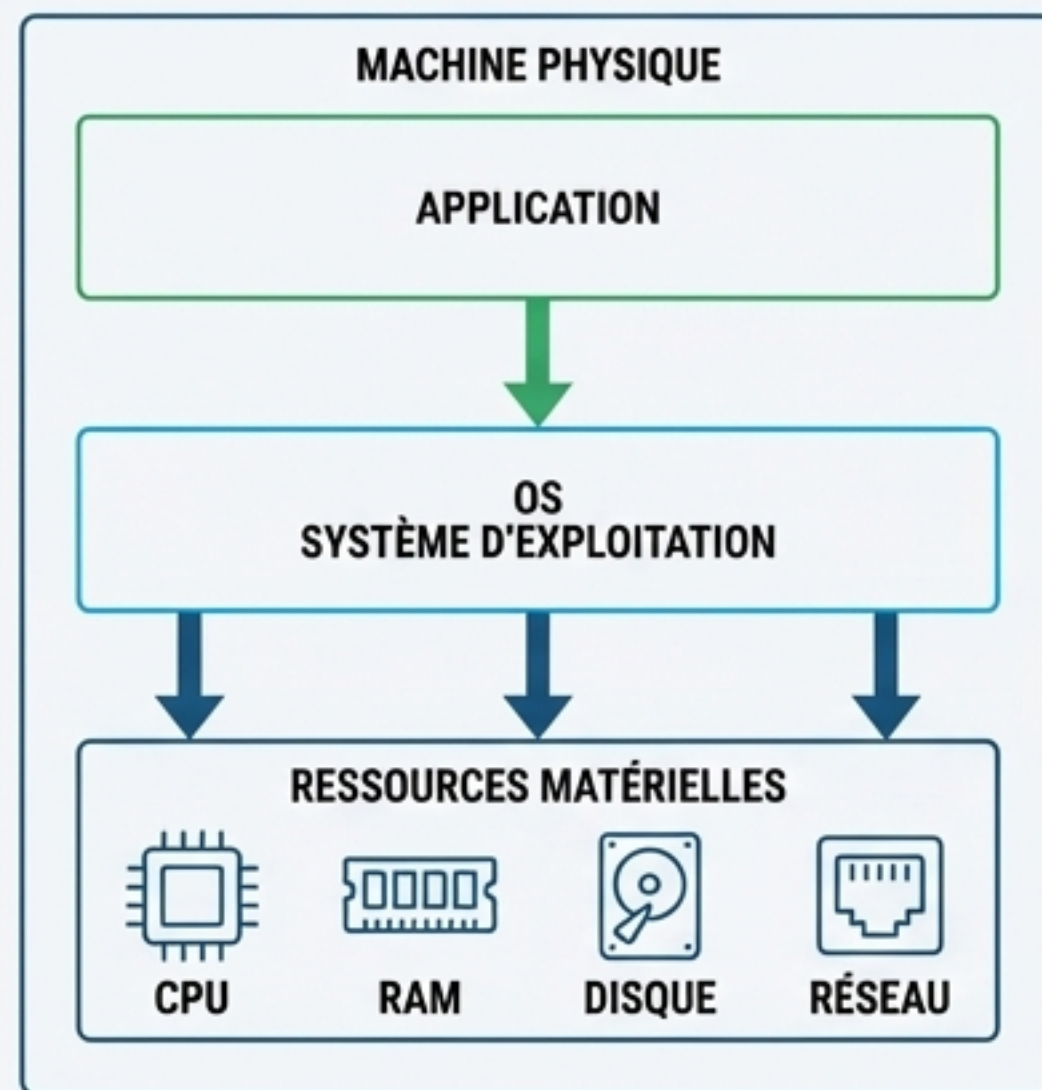


Pratique & Déploiement Agile
Fira Code (for any code mentions)

Le Point de Départ : La Virtualisation Classique

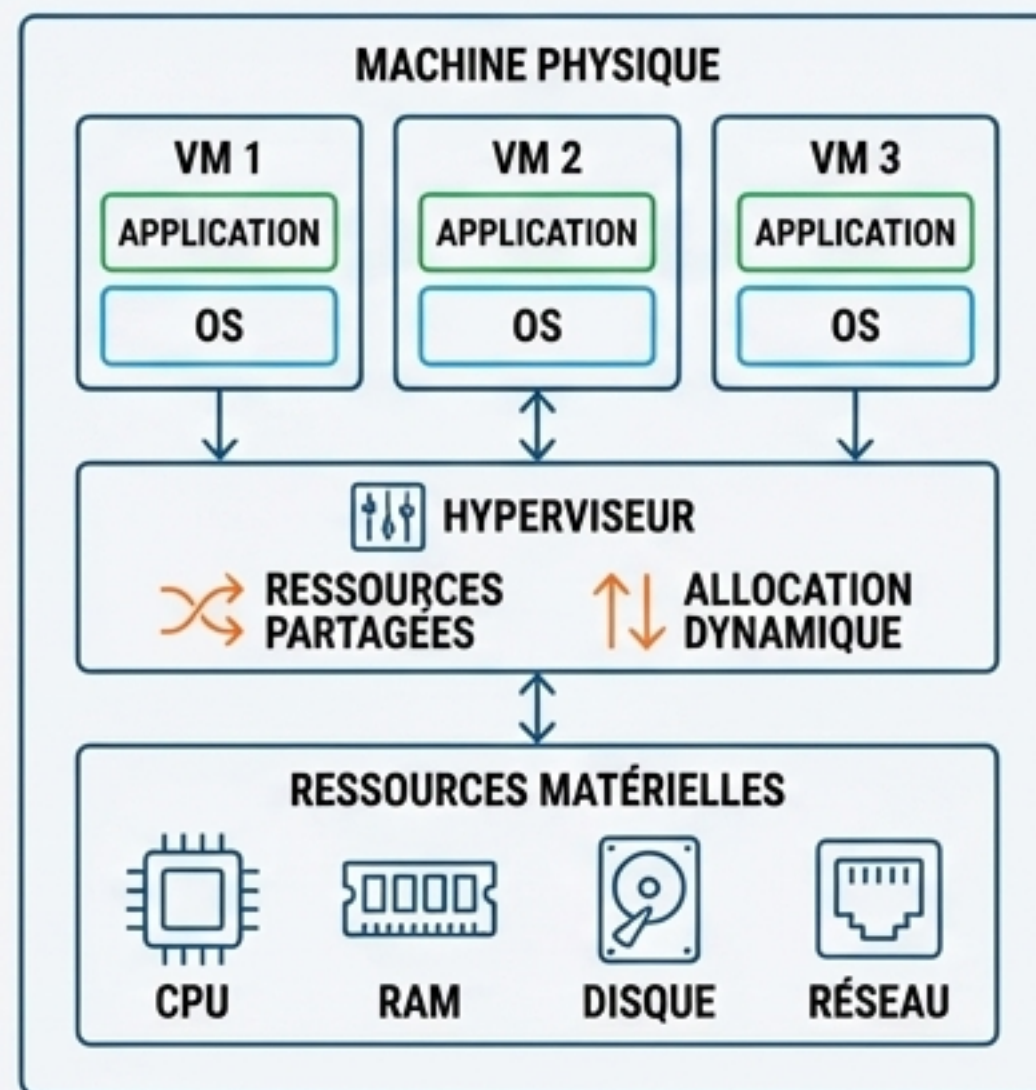
La virtualisation permet de faire fonctionner plusieurs systèmes d'exploitation (OS) complets sur un même ordinateur physique. Chaque système est une **"Machine Virtuelle"** (VM) qui agit comme si elle était indépendante.

Machine physique (sans virtualisation)



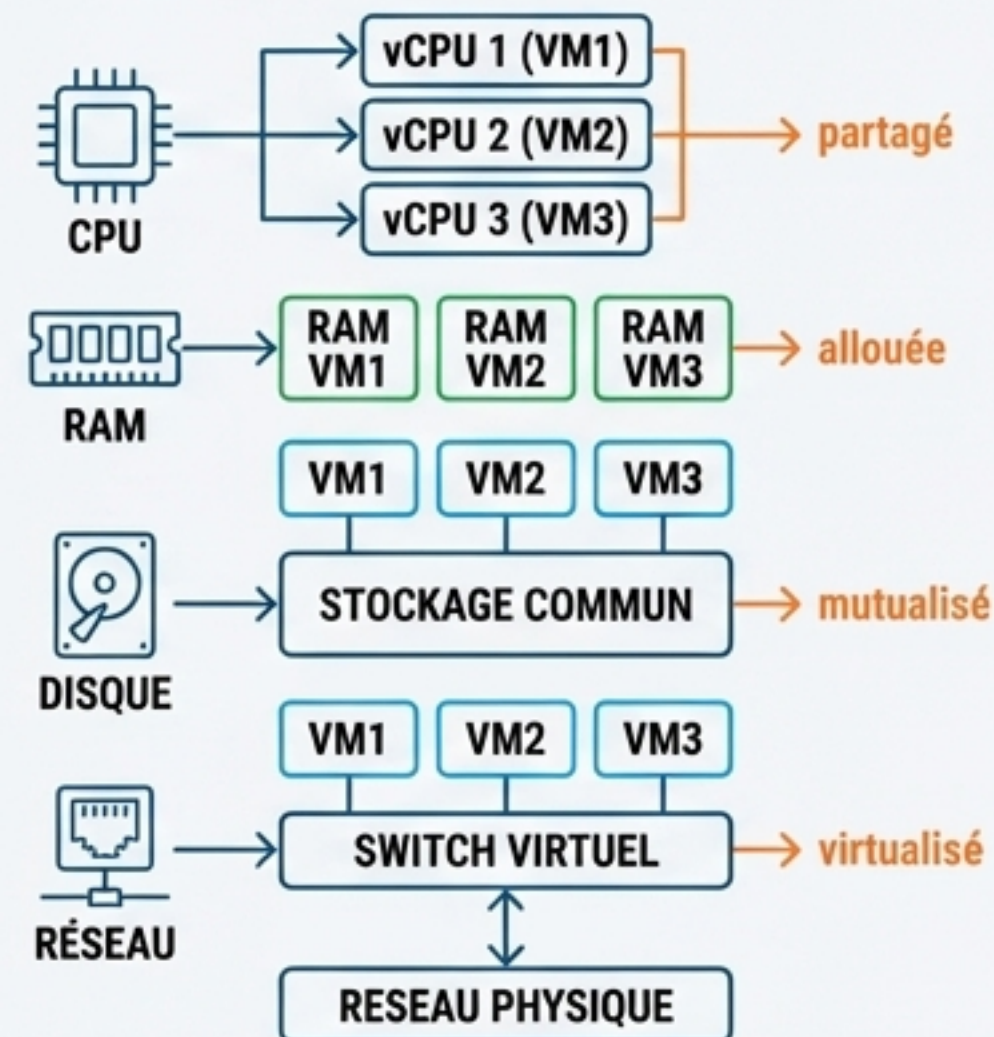
Machine Physique : Une seule application utilise directement les ressources de la machine via le système d'exploitation.

Machine physique virtualisée



Machine Virtualisée : L'hyperviseur partage et répartit les ressources matérielles entre plusieurs machines virtuelles.

Partage des ressources entre les VMs

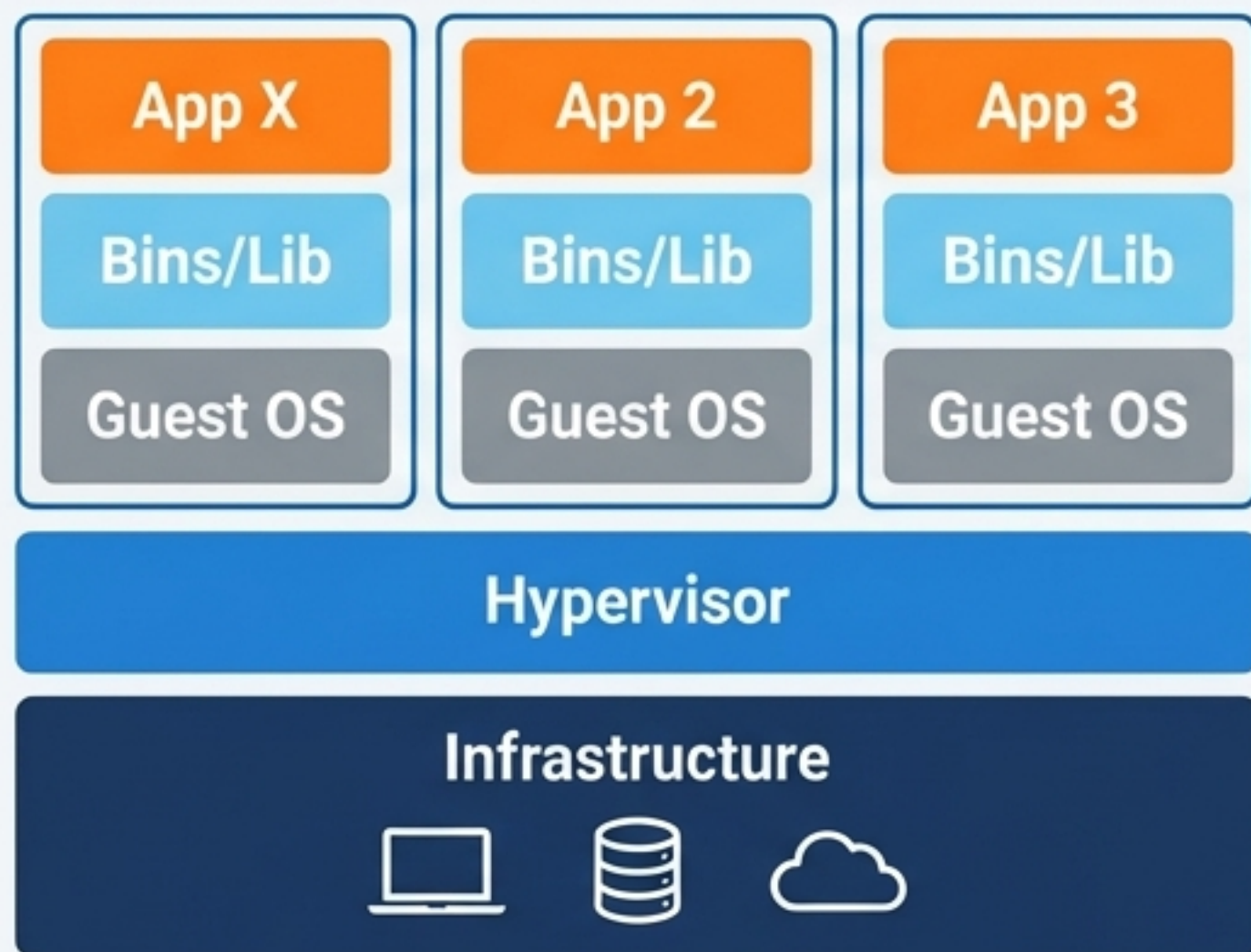


Partage des Ressources : Les VMs partagent les ressources physiques (CPU, RAM, Disque, Réseau) tout en restant isolées les unes des autres.

Une Révolution : L'Approche par Conteneurs

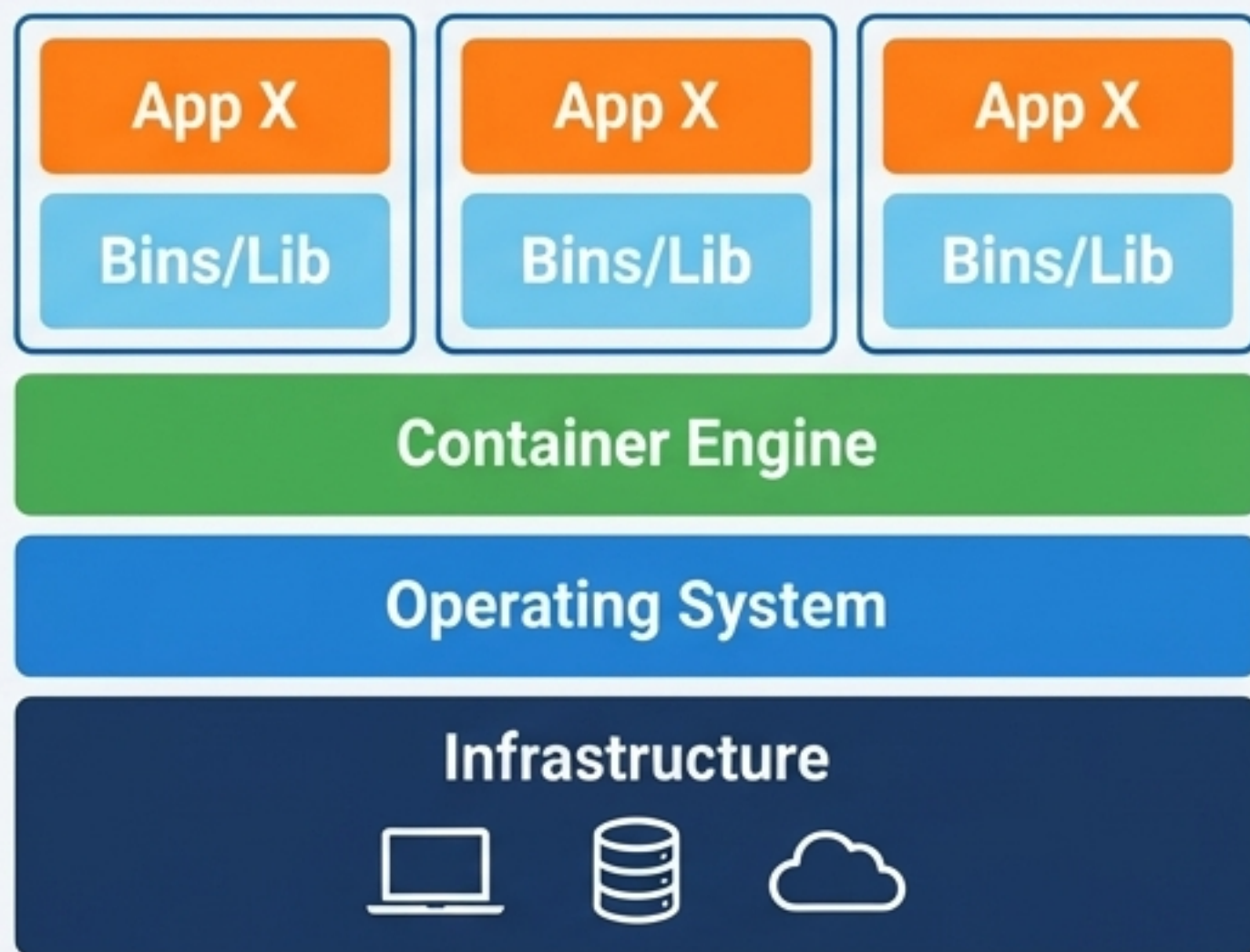
Docker propose une solution plus légère. Au lieu de virtualiser le matériel, il virtualise le système d'exploitation. Les conteneurs partagent le même noyau de l'OS hôte, rendant le tout plus rapide et efficace.

Virtual Machines



Chaque application a son propre OS invité ('Guest OS'), ce qui alourdit l'ensemble.

Containers



Toutes les applications partagent le même 'Operating System' et 'Container Engine' de l'hôte. **Pas d'OS invité.**

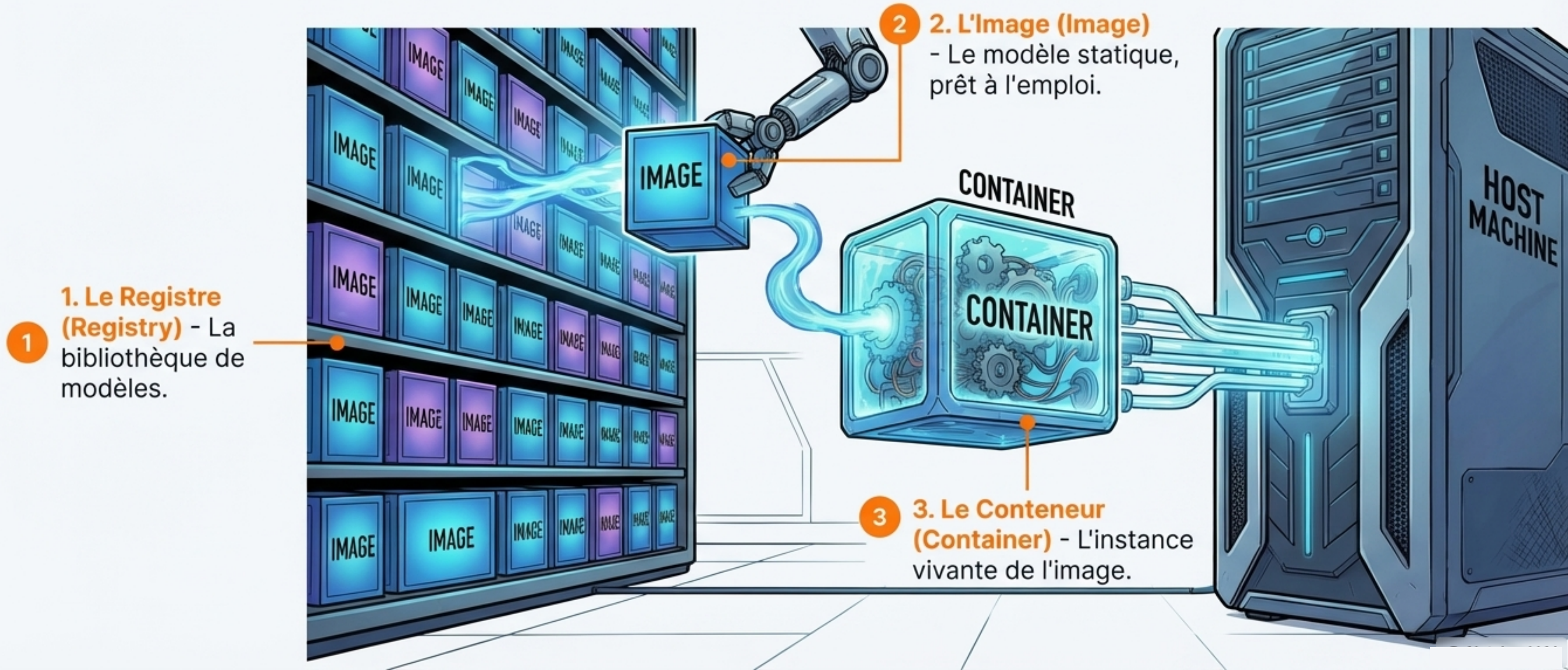
Machine Virtuelle vs. Conteneur Docker : Le Verdict

	 Machine Virtuelle (VM)	 Conteneur Docker
OS	OS complet (Guest)	Pas d'OS invité
Noyau (Kernel)	Propre à chaque VM	Partagé avec l'hôte
Taille	Lourd (plusieurs Go)	Léger (quelques Mo à centaines)
Démarrage	Lent (secondes/minutes)	Très rapide (ms/secondes)
Isolation	Forte (Hyperviseur)	Moyenne (namespace + cgroup)
Outils	Hyperviseur (VirtualBox, etc.)	Docker Engine

*Docker est idéal pour des environnements légers, rapides et reproductibles.
Les VMs restent utiles pour une isolation complète des systèmes.*

Le Cycle de Vie Docker : De la Recette à la Réalité

Le fonctionnement de Docker repose sur trois piliers : le Registre, l'Image et le Conteneur. Ensemble, ils forment un cycle de vie complet, de la conception à l'exécution.



Pilier n°1 : Le Registre, la Bibliothèque d'Images

Un registre est une bibliothèque centralisée où sont stockées les images Docker. Le plus connu est Docker Hub, mais il en existe d'autres (privés ou publics).

Convention de nommage

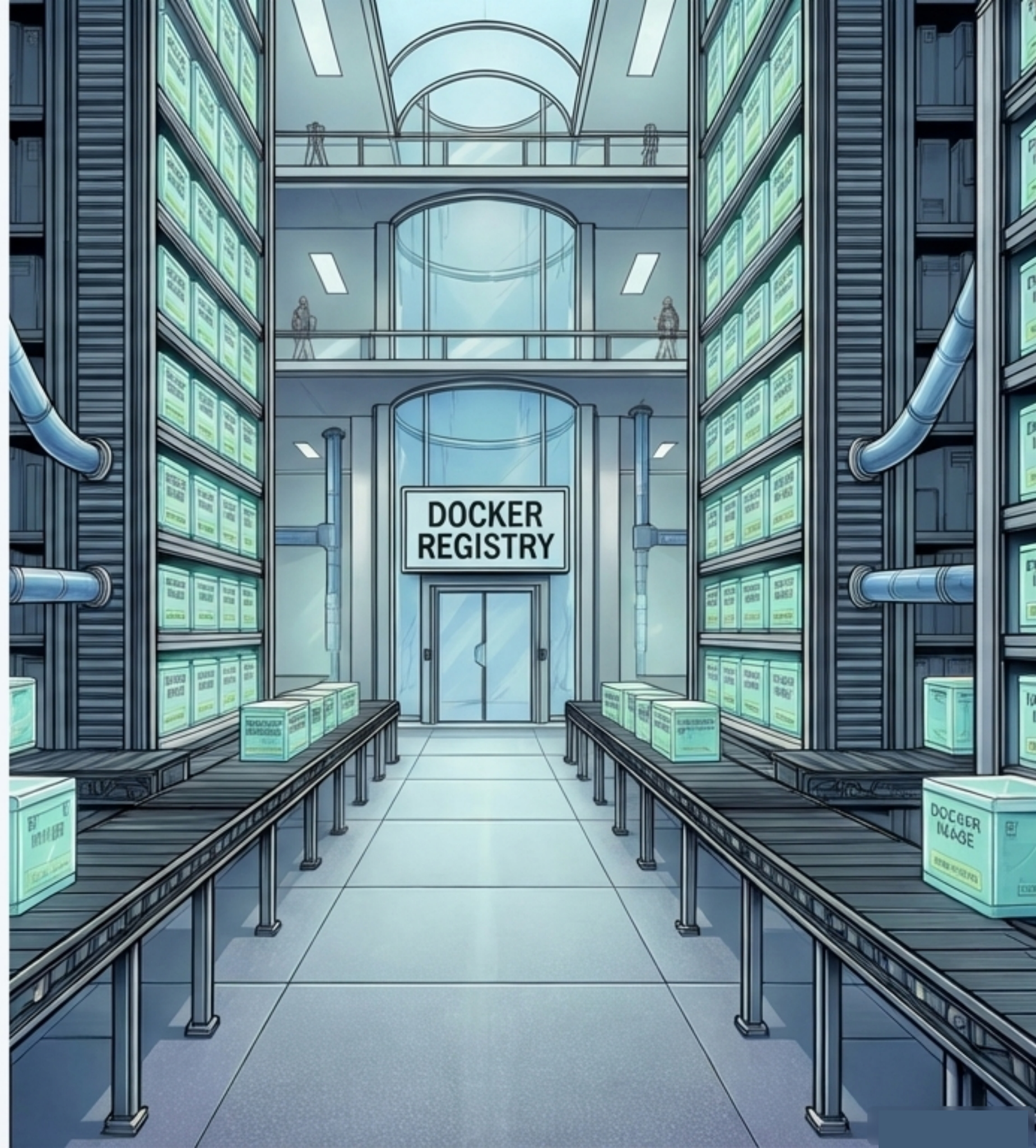
```
<nom-utilisateur>/<nom-image>:<tag>
```

nginx:latest

Image officielle, version la plus récente

sam/app-node:1.0

Image personnalisée de l'utilisateur 'sam', version 1.0



Pilier n°2 : L'Image, la Recette de votre Application

Une image est un modèle statique, en lecture seule (read-only), qui contient tout le nécessaire pour exécuter une application : un système minimal, les dépendances et le code applicatif. C'est une 'photo' prête à l'emploi, construite en couches.

Commandes clés

```
docker pull nginx
```

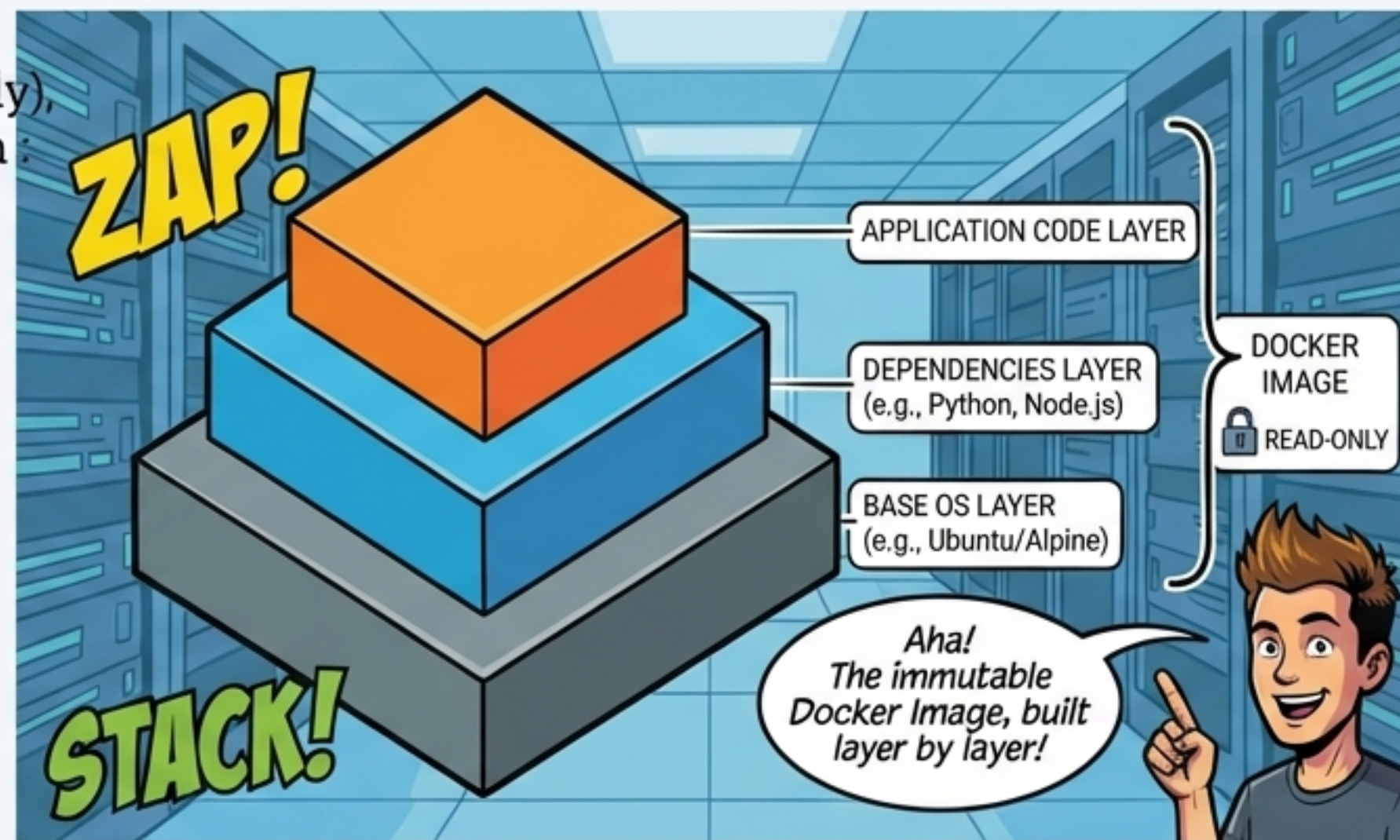
Pour télécharger une image depuis un registre.

```
docker images
```

Pour lister les images sur votre machine.

```
docker inspect nginx
```

Pour obtenir des détails techniques sur une image.



Pilier n°3 : Le Conteneur, l'Exécution Vivante de l'Image

Un conteneur est une instance vivante et exécutable d'une image. Lorsque le moteur Docker lance une image, elle devient 'vivante'. Chaque conteneur est isolé avec son propre système de fichiers, réseau et processus, et possède une couche inscriptible (read-write) pour les modifications.

Commandes clés

```
docker run nginx
```

Pour créer et démarrer un conteneur depuis une image.

```
docker ps
```

Pour lister les conteneurs en cours d'exécution.

```
docker exec -it <id> sh
```

Pour accéder au terminal d'un conteneur actif.



En Pratique : Commandes Essentielles



Gérer les Images

Rechercher : `docker search nginx`

Télécharger : `docker pull alpine`

Lister : `docker images`

Supprimer : `docker rmi nginx`



Gérer les Conteneurs

Exécuter : `docker run --name web nginx`

Lister (actifs) : `docker ps`

Lister (tous) : `docker ps -a`

Arrêter : `docker stop <id>`

Supprimer : `docker rm <id>`

Voir les logs : `docker logs <id>`



Nettoyage

`docker container prune`

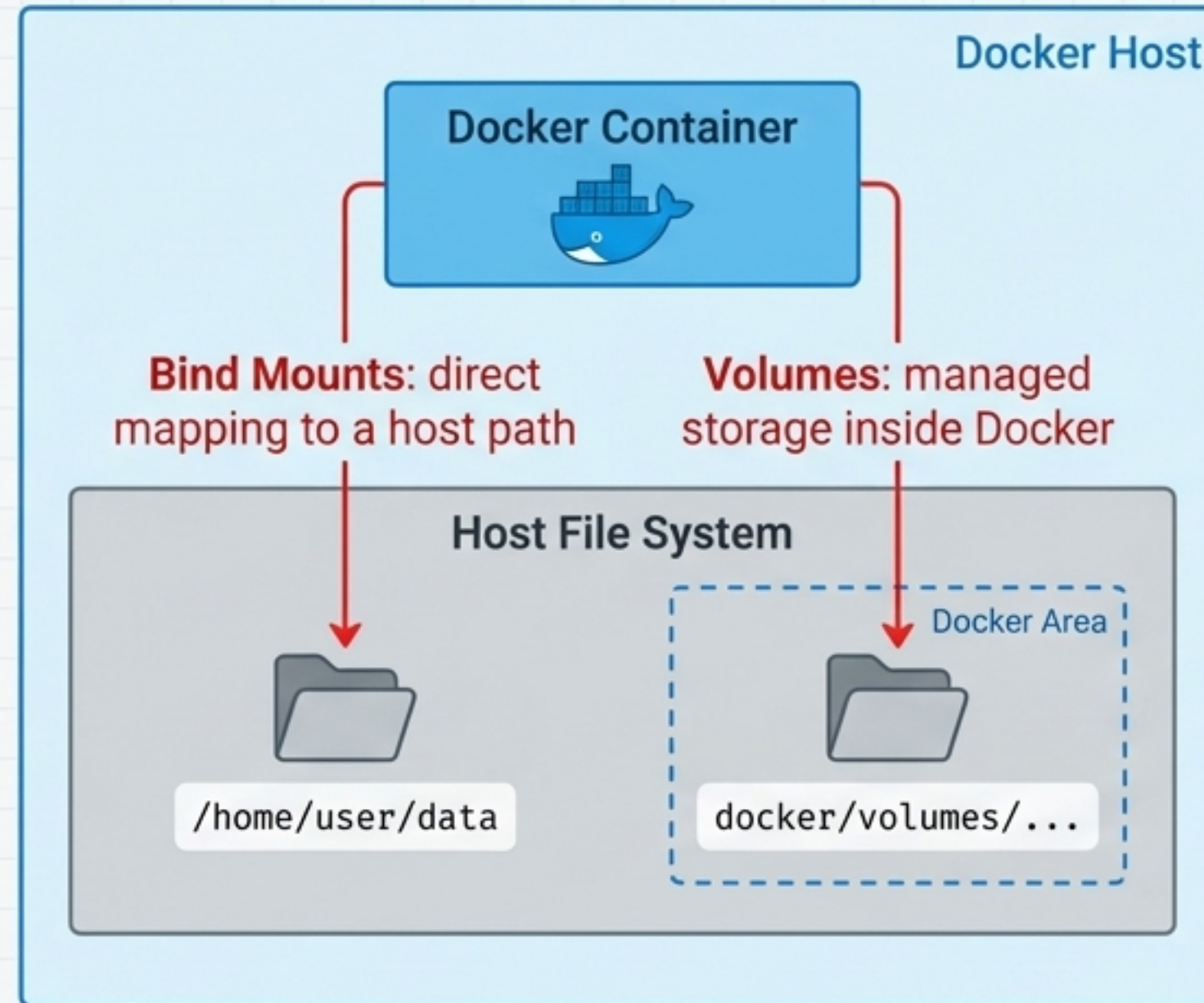
`docker image prune`

`docker system prune -a` ⚠️ **dangereux!**

Le Défi de la Donnée : Comment Rendre les Conteneurs Persistants ?

Le Problème

Par nature, les conteneurs sont éphémères. Si vous supprimez un conteneur de base de données, toutes les données qu'il contient sont perdues. Comment séparer le cycle de vie des données de celui du conteneur ?



Bind Mount = host directory directly linked to container.
Volume = storage managed by Docker, isolated from host path.

La Solution

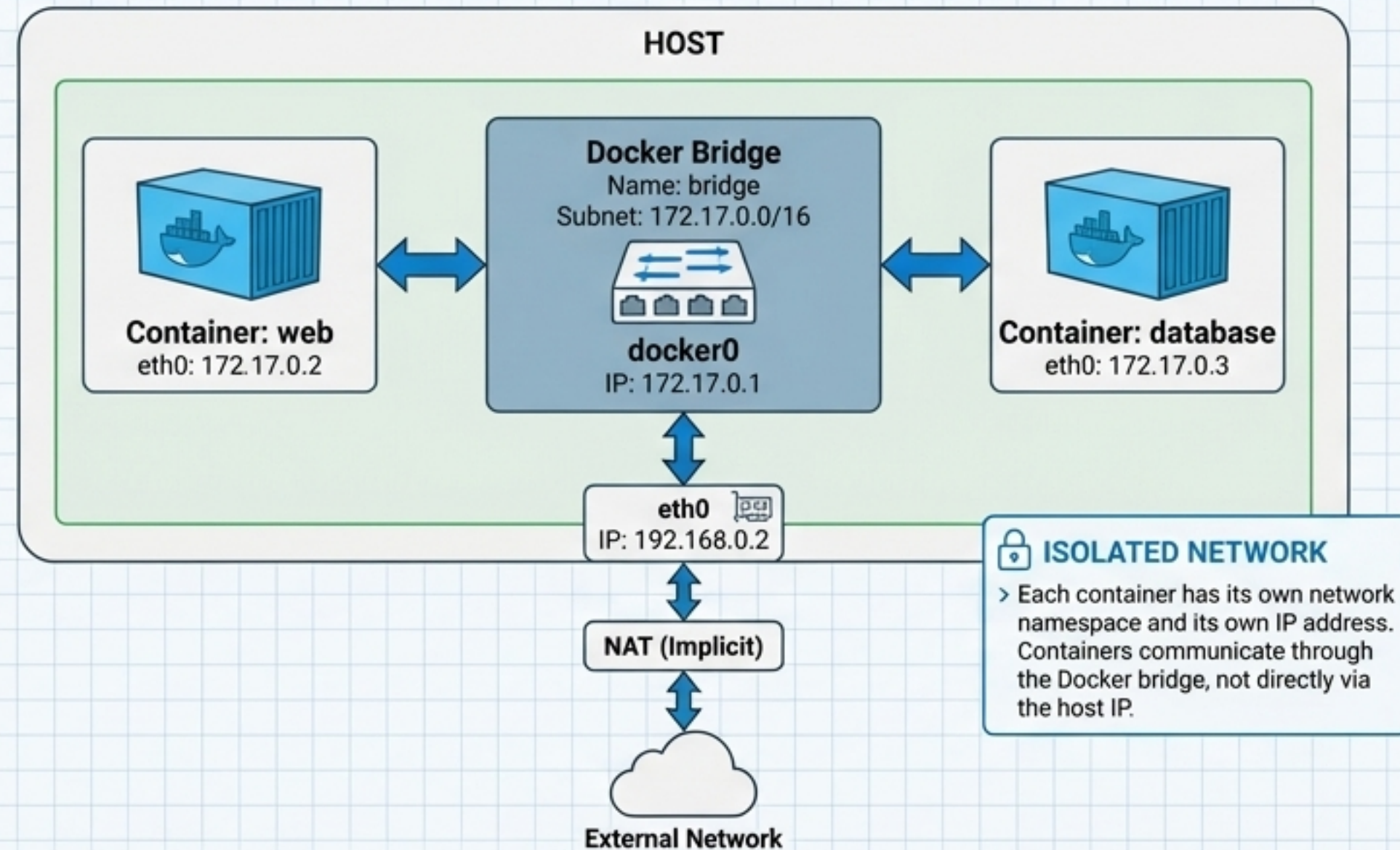
Docker propose deux mécanismes pour stocker et partager des données de manière durable : les **Volumes** et les **Bind Mounts**.

Volumes vs. Bind Mounts : Quelle Stratégie Choisir ?

Critère	Volumes	Bind Mounts
Gestion	Gérés par Docker (`/var/lib/docker/volumes/`)	Liés à un chemin spécifique de l'hôte (`\$(pwd)/site`)
Idéal pour	Production, bases de données, données critiques.	Développement, tests locaux, édition de code en direct.
Exemple	<code>docker run -v mes_donnees:/data nginx</code>	<code>docker run -v \$(pwd)/site:/usr/share/nginx/html nginx</code>
Portabilité	Élevée (indépendant de l'hôte).	Faible (dépend de la structure de fichiers de l'hôte).
Performance	Souvent plus performants pour les I/O intensives.	Accès direct, mais peut être plus lent sur certains OS.

Connecter les Conteneurs : Le Réseau Docker

Docker crée des réseaux virtuels pour permettre aux conteneurs de communiquer entre eux et avec le monde extérieur de manière sécurisée. Par défaut, chaque conteneur est attaché à un réseau de type 'bridge'.



Points clés



Chaque conteneur a sa propre adresse IP (ex: 172.17.0.2).



Ils communiquent via le 'Docker Bridge' (docker0), qui agit comme un routeur virtuel.



Les conteneurs sur le même réseau peuvent s'appeler par leur nom (ex: **'ping web'** depuis un autre conteneur).

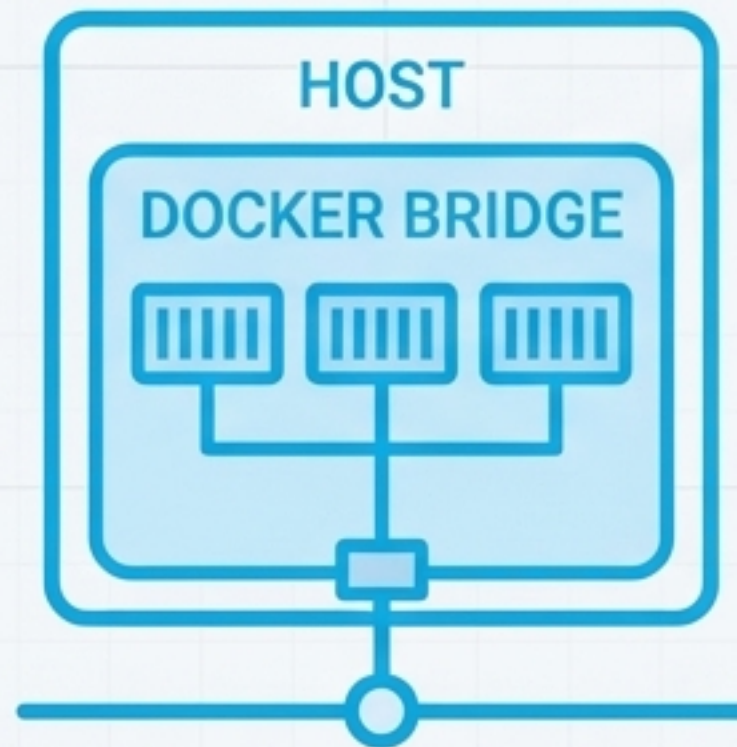
Modes Réseau : De l'Isolation à la Performance Maximale

Mode `bridge` (Par Défaut)

Réseau local isolé. Sécurité et flexibilité.

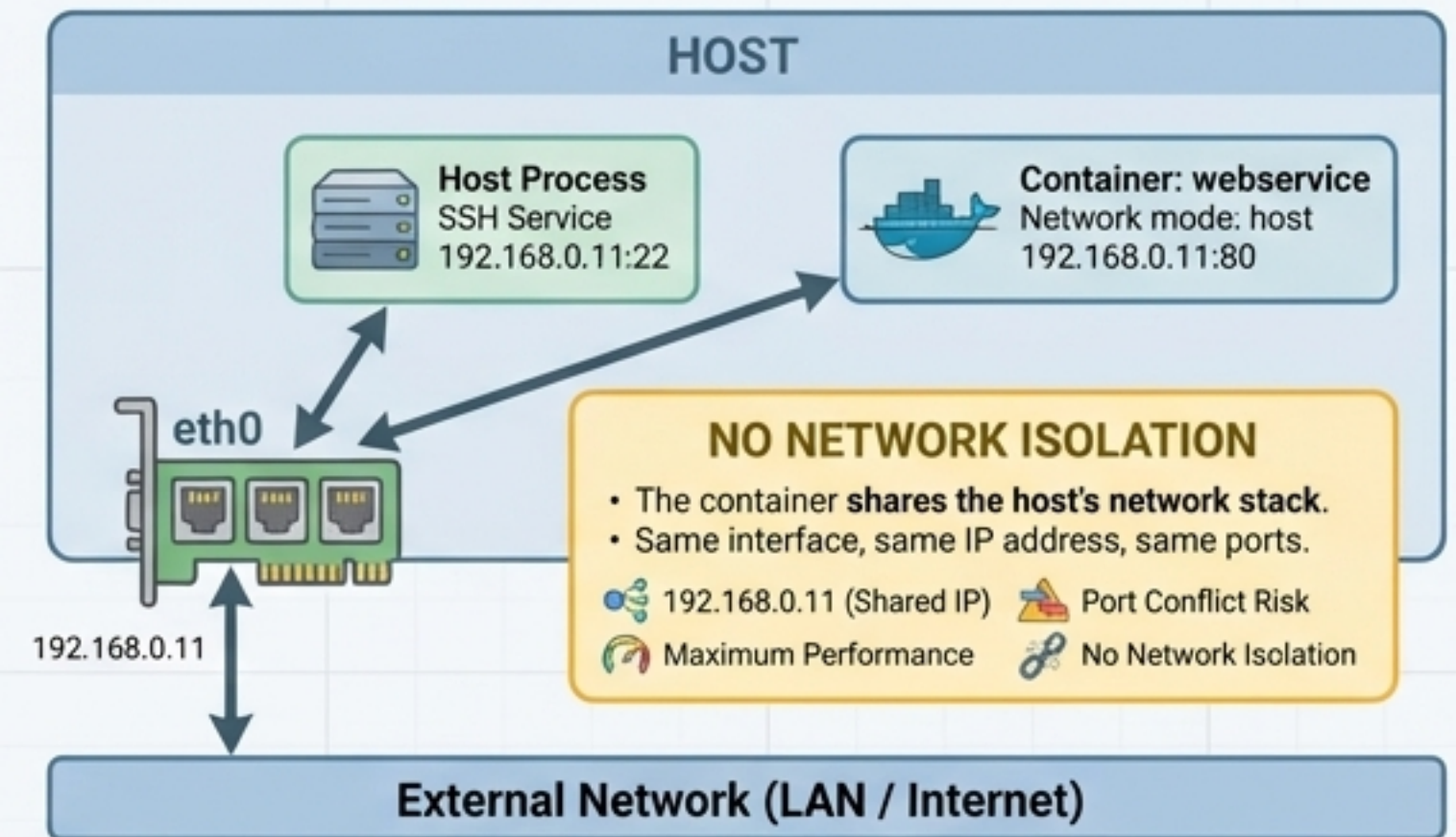
Cas d'usage :

La plupart des applications en développement et production.



Mode `host`

Le conteneur partage directement la pile réseau de l'hôte. Pas d'isolation réseau, pas d'IP propre.



- + Performance maximale (pas de NAT).
- Risque de conflits de ports, aucune isolation de sécurité.

Docker offre d'autres modes avancés (`overlay`, `macvlan`, etc.) pour des cas d'usage spécifiques comme les clusters ou l'intégration réseau complexe.

Cas Pratique : Faire Communiquer Deux Conteneurs

Nous allons créer un réseau isolé, y lancer un serveur web Nginx, puis lancer un autre conteneur pour vérifier la connectivité en utilisant le nom du service.

1 Créer un réseau personnalisé

```
docker network create monreseau
```

2 Lancer le conteneur serveur web sur ce réseau

```
docker run -d --network=monreseau --name web nginx
```

← Le conteneur s'appelle `web` et est attaché à `monreseau`.

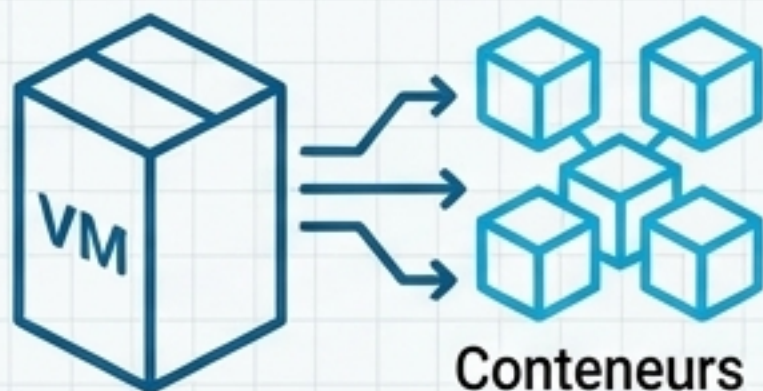
3 Lancer un conteneur de test et 'pinger' le serveur par son nom

```
docker run -it --rm --network=monreseau alpine ping web
```

← On peut atteindre `web` par son nom grâce au DNS interne de Docker.

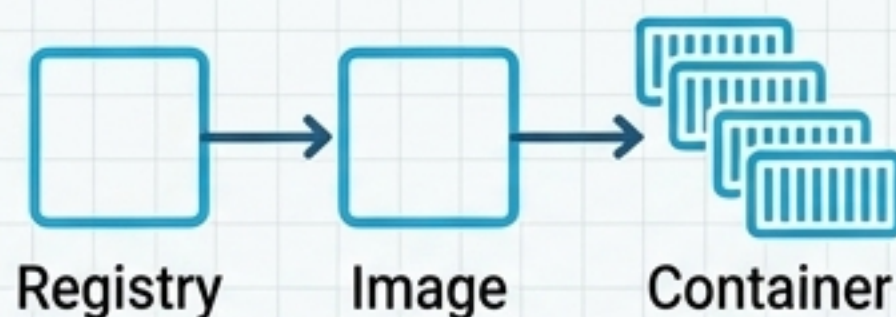
Docker : Votre Boîte à Outils pour le Développement Moderne

Le "Pourquoi"



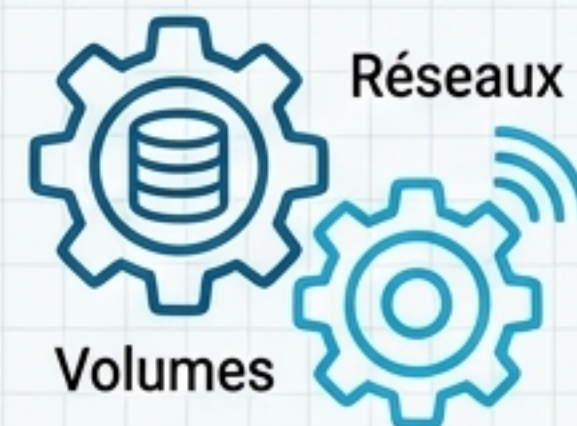
Passer de machines virtuelles lourdes à des conteneurs légers, rapides et portables.

Le "Quoi"



Orchestrer le cycle de vie de vos applications avec des modèles (Images) et des instances (Conteneurs).

Le "Comment"



Gérer les données persistantes (Volumes) et la communication (Réseaux) pour construire des applications robustes.

Vous avez maintenant les clés pour commencer à construire, distribuer et exécuter vos applications n'importe où.